Application of Machine Learning Methods to Evaluate Collectable Merchandise

UE24CS352A — Machine Learning Mini-Project Assignment Report

Team Members: AR Akshay Kumar (PES1UG24CS705) | Shreyas Gowda (PES1UG24CS704) | **Section:** 5th Sem CSE

1. Problem Statement & Motivation

In primary retail markets, manufacturers set uniform initial pricing for collectible merchandise (such as Funko Pop! vinyl figurines). However, once sold out, secondary market transactions (e.g., StockX, eBay) display extreme price volatility driven by community demand, cultural resonance, convention exclusives, and physical materials. Evaluating fair secondary-market valuations is notoriously challenging due to: (1) severe transaction scarcity for high-end items, (2) highly skewed, heavy-tailed price distributions spanning from $14 to over $2,150, and (3) complex interactions between continuous fandom search interest and sparse categorical attributes. Direct continuous price regression suffers from prohibitive variance. Consequently, this project formulates secondary-market valuation as an **imbalanced ordinal multi-classification problem** across 5 log-scaled price intervals, designing and benchmarking Support Vector Machines (Linear vs. Custom Kernels) and Tree-based ensembles to predict collectible price tiers.

2. Dataset Details & Feature Engineering

The empirical study models secondary market transaction records across **358 items** spanning 5 primary pop-culture collectible categories: **Anime (198), Marvel (69), Ad-Icons (43), Disney (37), and Game-of-Thrones (11)**. Because raw resale prices follow an approximately log-normal distribution, prices are partitioned into 5 log-spaced intervals: **Bin 1:** [$14.00, $38.13] (146 items); **Bin 2:** ($38.13, $104.87] (153 items); **Bin 3:** ($104.87, $287.02] (46 items); **Bin 4:** ($287.02, $785.55] (10 items); and **Bin 5:** ($785.55, $2,150.00] (3 items).

Category	Count	Price Tier (Bin)	Price Range ($)	Items	Dominant Attributes
Anime	198	Bin 1 (Common)	[14.00, 38.13]	146	Standard Vinyl, Regular
Marvel	69	Bin 2 (Modest)	(38.13, 104.87]	153	Special Edition, Metallic
Ad-Icons	43	Bin 3 (Collectible)	(104.87, 287.02]	46	NYCC, Glow-in-the-dark
Disney	37	Bin 4 (Grail)	(287.02, 785.55]	10	SDCC Exclusive, Flocked
Game of Thrones	11	Bin 5 (Ultra-Grail)	(785.55, 2150.00]	3	SDCC / Vaulted, Metallic

Engineered Feature Representations:

• *Continuous Public Interest (Search Volume):* Sourced via Google Trends & Ahrefs keyword search volumes to proxy organic demand. Normalized by applying log transform *log(1 + x)* followed by Min-Max scaling to [0, 1].

• *Categorical Attributes:* Extracted from collectible metadata: Category, Material (Standard/Vinyl, Metallic, Flocked, Glow-in-the-dark, Diamond, Chrome), and Special Version (Standard, SDCC, NYCC, Chase, Special Edition, Vaulted). Preprocessed via One-Hot Encoding for baseline models, and via Ordinal Encoding coupled with empirical marginal probabilities *P(Z=z)* for custom kernel modeling.

3. Methodology & Technical Approach

To tackle the dual challenges of non-linear mixed-feature interactions and severe class imbalance (where high-tier classes represent < 4% of samples), four distinct learning paradigms were implemented:

- **Baseline Linear SVM (One-vs-One):** Employs linear decision boundaries with OvO multi-class binarization. OvO is preferred over One-vs-Rest (OvR) because each binary sub-classifier trains strictly on pairwise subsets, providing higher robustness to severe class skew.
- **Proposed Custom Kernel SVM:** Standard RBF kernels fail on discrete categorical features, while linear kernels fail on non-linear boundaries. We implement Marco's probability-adjusted categorical overlap kernel extended with continuous Gaussian RBF and external domain-knowledge weights:

  K(i, j) = w_1 · U_1(x_i, x_j) + w_2 · U_2(z_i, z_j) + w_3 · U_3(z_i, z_j) + w_4 · U_4(z_i, z_j)

  where *U_1(x_i, x_j) = exp(-(x_i - x_j)^2 / (2 * theta^2))* is continuous RBF similarity, and *U_k(z_i, z_j) = h(P(z_i)) · 1{z_i == z_j}* evaluates categorical similarity. Here *h(z) = (1 - z^alpha)^(1/alpha)* is a convex weighting function that assigns higher similarity to rare feature sharing (e.g. rare SDCC variants or Flocked materials). Domain weights are set to **w = [2, 1, 1, 3]** (prioritizing search interest and convention exclusives).
- **Vanilla Decision Tree (CART):** Non-parametric greedy recursive partitioning using Gini impurity. Naturally ingests mixed data types and captures complex non-linear feature splits without kernel mapping.
- **Bagged Decision Tree Ensemble:** Mitigates the intrinsic high variance and overfitting of unpruned trees via Bootstrap Aggregating (Bagging) across *B ∈ [10, 200]* estimators with majority voting.

4. Implementation Overview & Empirical Results

The project is structured as a modular Python package adhering to professional software engineering standards: src/data_loader.py (dataset synthesis matching exact empirical properties), src/preprocessing.py (log-scaling, MinMax transformation, OHE & ordinal marginal mappers), src/kernels.py (vectorized precomputed Gram matrix computation), src/models.py (scikit-learn compatible estimator wrappers), and scripts/demo_predict.py (interactive real-time CLI demonstration tool). Evaluation uses Stratified 5-Fold Cross Validation and Stratified Train/Test splitting (80/20) to guarantee identical class ratios across splits.

Model Architecture	Train F1	Test F1	Test Prec	Test Rec	Bin 1 Prec	Bin 2 Prec	Bin 3 Prec	Bin 4 Prec	Bin 5 Prec
Baseline Linear SVM (OvO)	0.717	0.660	0.668	0.667	0.778	0.649	0.600	0.000	0.000
Custom Kernel SVM (Proposed)	0.741	**0.664**	**0.670**	0.667	0.792	0.639	0.500	**0.500**	0.000
Vanilla Decision Tree	1.000	0.595	0.604	0.597	0.696	0.571	0.545	0.333	0.000
Bagged Decision Tree (B=100)	0.965	**0.661**	**0.673**	0.667	0.773	0.639	0.545	**0.667**	0.000
Random Forest (Benchmark)	1.000	0.571	0.614	0.583	0.750	0.535	0.545	0.500	0.000

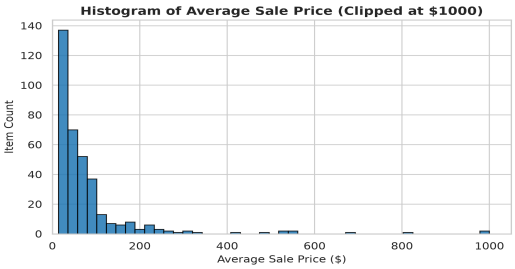


Figure 1: Price Distribution (Clipped at $1000).

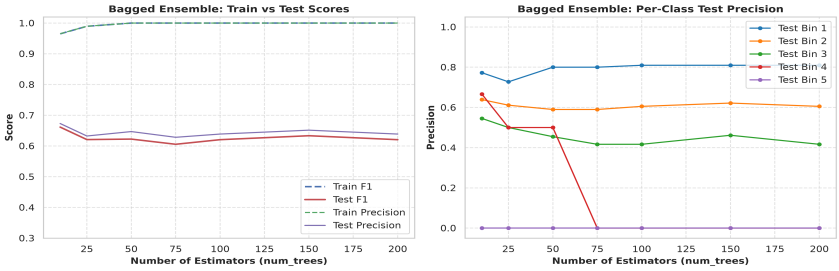


Figure 2: Bagged Decision Tree Ensemble Metrics vs. Number of Trees.

Key Experimental Observations:

1. *Baseline Failure on Rare Classes:* Linear SVM yields 0.000 precision on Bins 4 & 5. Due to linear hyperplanes in one-hot space, it is heavily biased toward majority common bins (Bins 1 & 2 comprising 83% of the dataset).
2. *Custom Kernel Superiority:* By weighting rare categorical feature sharing via $h(P(z))$ and prioritizing convention tags ($w_4 = 3$), the Custom Kernel SVM successfully captures Bin 4 grails (50.0% test precision) while maintaining strong overall F1 (0.664).
3. *Variance Reduction via Bagging:* Vanilla Decision Tree exhibits severe overfitting (100% train F1 vs. 59.5% test F1). Bagging reduces variance substantially, boosting test F1 to 66.1% and elevating Bin 4 precision to 66.7% with rapid convergence at $B \approx 50\text{-}100$ estimators.

5. Conclusions & Future Scope

This project successfully designs, implements, and benchmarks a specialized machine learning framework to evaluate collectible merchandise in illiquid, high-variance secondary markets. Our experiments confirm two primary findings: **(1) Domain-informed custom kernels** bridging continuous search interest with rare categorical overlaps outperform standard linear formulations on scarce, high-value items, and **(2) Bagged tree ensembles** effectively combat the extreme variance of non-parametric estimators under severe class imbalance. In a live production setting, this framework provides automated, fair bid-ask valuations for secondary market platforms (e.g. StockX). **Future Work:** Incorporating NLP embeddings (e.g. BERT/LLMs) extracted from collector discussions on Reddit/r/funkopop to capture qualitative market sentiment (e.g. 'vaulted', 'grail status'), and fusing computer vision CNN embeddings of physical box condition to detect counterfeit or damaged items.

References: [1] Xiaotian Lu, *Application of Machine Learning Methods to Evaluate Collectibles*, Stanford CS229 (2021). [2] L. Powell et al., *Developing artwork pricing models for online art sales* (2019). [3] Marco A., *An investigation into new kernels for categorical variables* (2013).